PRODUCTHUB

Task 1 – RESTful API Development

Node.js • Express.js • MongoDB • Mongoose

Submitted by: Vanshika Khandelwal

GitHub Repository: https://github.com/vanshika2723/ProductHub-REST-API.git

Submission Date: [15 October 2026]

1. Project Overview

ProductHub is a RESTful API for managing product inventory. It supports creating, retrieving, updating, and deleting product records. The API is built with Node.js and Express.js, while MongoDB stores product data and Mongoose defines the schema and validation.

2. Objectives

- Build REST endpoints using Express.js.
- Connect the application to MongoDB using Mongoose.
- Implement CRUD operations for product records.
- Validate incoming product data and handle errors consistently.
- Test API endpoints with Postman and document the results.

3. Technologies Used

Technology	Purpose
Node.js	JavaScript runtime for the backend
Express.js	HTTP server and routing
MongoDB	Database for product records
Mongoose	Schema, validation, and database operations
dotenv	Loads configuration from environment variables
Postman	Manual API endpoint testing

4. Project Structure

```
ProductHub-REST-API/
├── config/db.js
├── controllers/productController.js
├── middleware/errorMiddleware.js
├── models/Product.js
├── routes/productRoutes.js
├── postman/ProductHub.postman_collection.json
├── .env.example
├── .gitignore
├── package.json
├── package-lock.json
```

```
├── README.md
└── server.js
```

5. Setup Commands

```
mkdir ProductHub-REST-API

cd ProductHub-REST-API

npm init -y

npm install express mongoose dotenv

npm install --save-dev nodemon

npm run dev
```

Configuration note: create a local .env file with PORT, MONGO_URI, and NODE_ENV. Do not upload real credentials or the .env file to a public repository.

6. API Endpoints

Method	Endpoint	Purpose
GET	/	API health check
GET	/api/products	Retrieve all products
GET	/api/products/:id	Retrieve one product
POST	/api/products	Create a product
PUT	/api/products/:id	Update/replace product
PATCH	/api/products/:id	Partially update a product
DELETE	/api/products/:id	Delete a product

7. Sample Request Body

```
{
  "name": "Wireless Mouse",
  "description": "Ergonomic wireless mouse",
  "price": 799,
  "category": "Electronics",
  "stock": 25
}
```

8. Key Implementation Details

The Product Mongoose schema defines required fields, text length limits, non-negative price and stock values, integer stock validation, an active flag, and timestamps. Controllers implement CRUD operations. Routes expose the endpoints under /api/products. Centralized error middleware handles validation errors, invalid IDs, duplicate-key errors, and unexpected server errors.

9. Testing Evidence – Add Your Screenshots

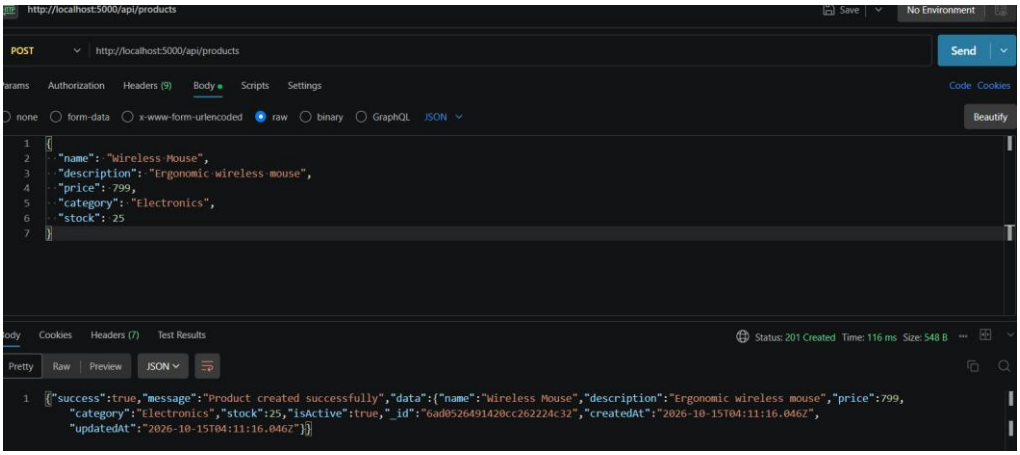
Replace each placeholder below with a real screenshot captured from your running application. Do not claim a test passed unless you have run it and verified the response.

Screenshot 1: Server and MongoDB connection

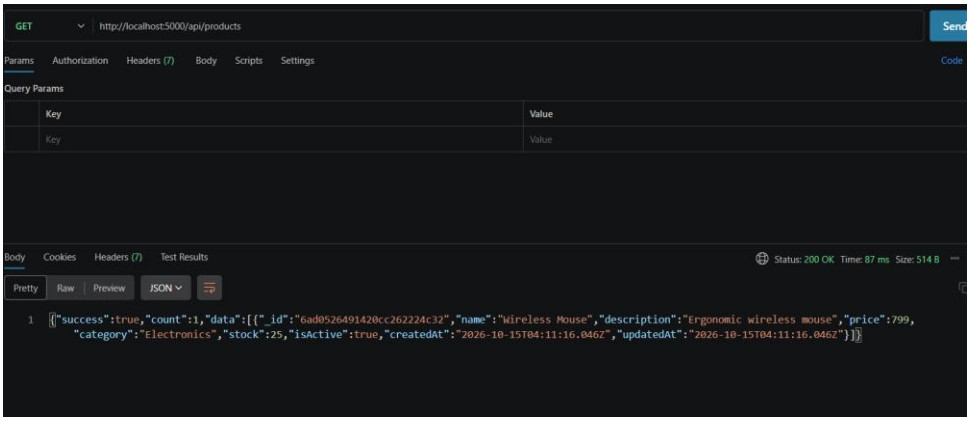
Paste screenshot of terminal showing the server started and database connected.



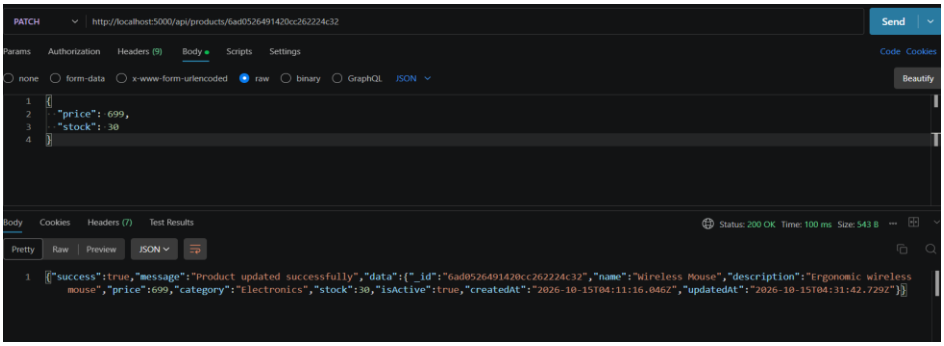
Screenshot 2: Create product (POST)



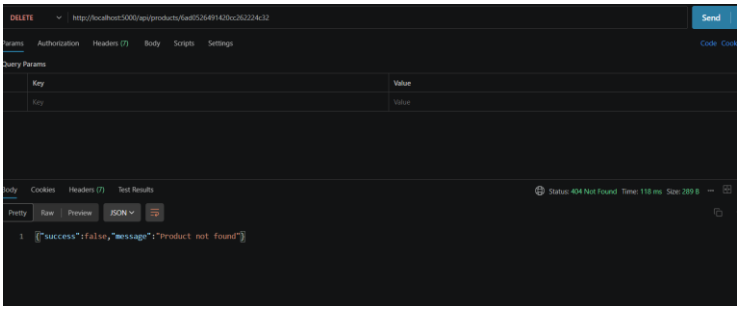
Screenshot 3: Get all products (GET)



Screenshot 4: Update product (PUT or PATCH)

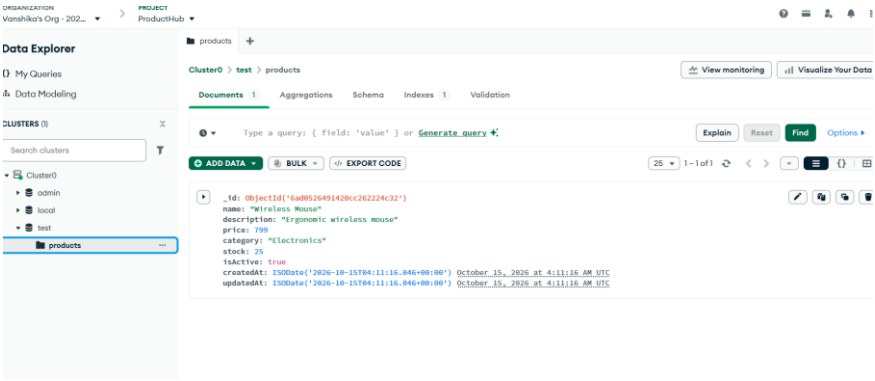


Screenshot 5: Delete product (DELETE)



Screenshot 6: MongoDB data

Paste screenshot from MongoDB Compass or Atlas showing saved product records.



10. Repository and Submission Links

GitHub Repository: https://github.com/vanshika2723/ProductHub-REST-API.git

Postman Collection: Included in the repository at postman/ProductHub.postman_collection.json

11. Conclusion

ProductHub demonstrates the core principles of REST API development using Node.js, Express.js, and MongoDB. The project organizes routes, controllers, models, and middleware separately and provides CRUD endpoints with schema validation and centralized error handling. Complete this report with actual test screenshots and the final public repository link before submission.